Hardware–Software Co-Design of a Multiplier-Less SiLU Custom Instruction for RISC-V: In-Order versus Out-of-Order in gem5

Ayush Krishna
School of Electronics Engineering, VIT Vellore
Course BAEVD202, Computer Architecture
ayushkrishb1@gmail.com

***Abstract*—The Sigmoid Linear Unit, $\mathrm{SiLU}(x) = x\,\sigma(x)$, is the activation of YOLOv8 and many other networks, and on a core without hardware exponentials it is computed by an expensive library call. We take a published multiplier-free piecewise-linear approximation (six segments, shifts and adds only), implement it as a combinational Verilog unit, synthesise it for an FPGA, and add it to gem5 as a RISC-V custom instruction whose latency is *derived from the synthesis delay* by a script rather than typed by hand. Using the same binaries on gem5's in-order (MinorCPU) and out-of-order (O3CPU) cores, we measure the iron-law terms, the effect of functional-unit latency and pipelining, branch mispredictions of the software version, code-placement sensitivity, and cache pressure. We then swap the approximation into YOLOv8n and measure accuracy and an Amdahl-bounded whole-network estimate. On an activation loop the instruction is 24× (in-order) and 36× (out-of-order) faster than the C library `expf`, but only 2.4× and 3.7× faster than the same approximation written in integer C. On a whole network running on a scalar core the gain over `expf` is about 1.1× and over integer C about 1.01–1.02×; it becomes 2.5–2.7× over `expf` only if the multiply-accumulate work is made 16× faster. The approximation costs about 0.0015 mAP50–95 beyond fixed-point rounding on a 128-image set. The out-of-order benefit exists only if the unit is pipelined: an unpipelined 5-cycle unit is slower than integer C.**

***Index Terms*—RISC-V, custom instruction, SiLU, gem5, Amdahl's law, out-of-order execution, hardware–software co-design**

I. Introduction

Activation functions are a small part of a neural network's arithmetic but are awkward on simple processors: SiLU [2], [3] needs an exponential and a division. A natural architectural reply is a custom instruction. Whether that pays off is a computer-architecture question, not a circuit question: it depends on how many instructions the baseline needs, on cycles per instruction (CPI) in the pipeline that executes it, on the clock period, and on what fraction of total run time the function occupies. This work therefore treats the instruction as an object to be evaluated with the quantitative tools of the course — the iron law, Amdahl's law, hazards, in-order versus out-of-order execution, and branch prediction — rather than as a hardware block to be admired.

Contributions. (1) A complete, scripted flow from RTL and timing report to a gem5 functional unit whose latency is computed (not hand-entered) from the synthesis delay and the target clock. (2) A fair comparison of five code variants (loop only, C `expf`, integer approximation, 257-entry table, custom instruction) on two core models, across clocks and latencies, with branch, alignment, and cache controls. (3) A model-level evaluation on YOLOv8n that separates accuracy from speed and bounds the speed benefit by Amdahl's law. (4) A frank list of what these simulations cannot show.

II. Background

A. The approximation

Spagnolo, Perri and Corsonello [1] propose piecewise-linear approximations of Swish (SiLU) that need only shifts and adds. We use their six-segment variant ("f_4" in our code) in Q6.10 fixed point (16-bit, 10 fractional bits, so $1.0 = 1024$). With x the 16-bit input:

$$f_4(x) = \begin{cases} 0 & x \le -4 \\ -x/16 - 0.375 & -4 < x \le -1 \\ x/4 - 0.037 & -1 < x \le 0 \\ 3x/4 - 0.037 & 0 < x \le 1 \\ x + x/16 - 0.375 & 1 < x \le 4 \\ x & x > 4 \end{cases} \tag{1}$$

where the divisions are arithmetic shifts and the constants are 384 and 38 in Q6.10. Only shifts, adds and compares are needed. Over $(-4, 4)$ the mean absolute error is 0.0279 (the paper reports 0.0278) and the maximum is 0.072 at the segment edges; $f_4(0) = -0.0375$ whereas $\mathrm{SiLU}(0) = 0$, and the output is discontinuous at $x = \pm 4$ (step 0.125) and $x = \pm 1$ (0.025).

B. Course concepts used

The *iron law* [6] states $T = \mathrm{IC} \times \mathrm{CPI} \times T_{clk}$: the instruction removes dynamic instructions (IC) and may change CPI and T_{clk}. *Amdahl's law* [7] bounds a whole-program speedup by the fraction of time sped up. A multi-cycle unit creates *data hazards* for dependent instructions and, if unpipelined, a *structural hazard*. An in-order pipeline stalls on these; an out-of-order core can overlap independent iterations. Software f_4 is a chain of data-dependent compares, so its performance depends on *branch prediction*.

III. Method

A. Golden model, RTL and synthesis

A Python model (`golden/f4_golden.py`) defines the reference output for all 65,536 inputs. The RTL (`rtl/silu.v`) is combinational: a range selector on the Q6.10 input, constant shifts and one adder per segment. It was synthesised with Quartus Prime 23.1std for a Cyclone IV E EP4CE115F29C7 with a 10 ns constraint: data delay 8.058 ns (slow 1200 mV, 85°C corner), setup slack +1.876 ns, $F_{max} \approx 123$ MHz, 191 logic elements, no embedded multipliers and no memory bits. The tool's power estimate (153 mW) is marked low-confidence (no toggle data) and is not used.

B. From delay to simulator latency

A latency typed into the simulator by hand would leave synthesis and simulation unconnected. Here `scripts/run_gem5.py` computes the unit latency from the delay and the simulated clock. For a multi-cycle (unregistered) unit, $L = \lceil d/T_{clk} \rceil$. For a pipelined unit, L is the smallest N with $d/N + 0.3\,\text{ns} \le T_{clk}$, where 0.3 ns is an *assumed* register overhead per stage; the pipelined unit accepts one operation per cycle, the multi-cycle unit is busy for L cycles. With $d = 8.058$ ns this gives $L = 1, 2, 5$ at 100, 200, 500 MHz, and $L = 12$ (pipelined) or 9 (multi-cycle) at 1 GHz.

C. gem5 integration

We used gem5 v23.0.0.1 [4], [5], RISC-V [9], syscall-emulation mode. The instruction is an R-type in the custom-0 opcode space (0x0B, funct3 = 0, funct7 = 0) computing `rd = f4(rs1); rs2` is unused. It is given its own operation class so that a dedicated functional unit can carry the latency and issue rate. A wrapper configuration (`gem5/configs/silu_se.py`) removes this class from the shared SIMD/floating-point units and adds the dedicated unit to both core models: `RiscvMinorCPU` (in-order) and `RiscvO3CPU` (out-of-order), otherwise at gem5 defaults. The instruction and the integer C version were also run through all 65,536 inputs inside gem5; together with the RTL, the C kernels and a PyTorch emulator all agree with the golden model with 0 mismatches.

D. Benchmarks

Five variants of one activation loop (1,024 inputs in $[-4, 4)$, one warm-up pass, 20 timed passes bracketed by m5 reset/dump instructions) are compiled from identical source with the same flags (`-O2 -static`, clang via Zig with musl, hard-float, `rv64gc`): *loop only* (load, accumulate, branch), C `expf` (SiLU computed from the library exponential), integer f_4 in C, a 257-entry interpolated table (514 bytes), and the custom instruction. A 3×3 convolution layer (8→8 channels, 16×16, 147,456 MACs, 2,048 activations, scalar code) serves as the model-level proxy. Loops are aligned to 64 bytes (`-falign-loops=64`) for the headline numbers.

TABLE I
One loop, 1,024 inputs, 100 MHz, unit latency 1. Cycles per activation (CPI in parentheses). Sorted inputs.

Variant	Instr./act.	In-order	Out-of-order
loop only	5.0	6.03 (1.20)	1.98 (0.39)
float `expf`	114.4	169.8 (1.48)	36.8 (0.32)
integer f_4	12.8	16.56 (1.30)	3.82 (0.30)
table (514 B)	21.0	21.03 (1.00)	5.02 (0.24)
custom insn	6.0	7.04 (1.17)	1.03 (0.17)

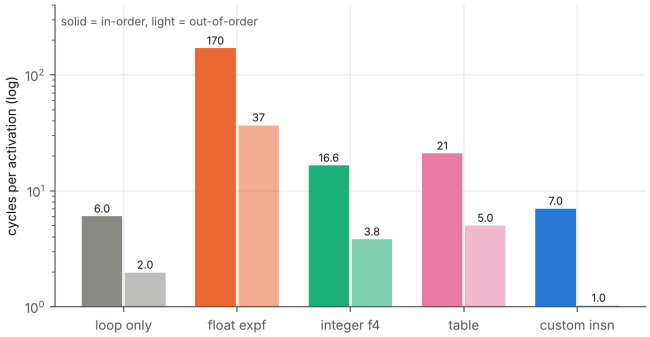


Fig. 1. Cycles per activation, five variants, two cores (100 MHz, sorted inputs).

E. Model-level evaluation

All 57 SiLU modules of YOLOv8n [8] (Ultralytics 8.4.183, PyTorch 2.14.1) are replaced and we verify that none remain and that the replacement is called (969 calls in the validation run $= 57 \times 17$ batches). Accuracy is measured on COCO128 (128 images, 929 instances), *relative to the exact SiLU only*; it is not a published mAP. The network has 4.37 GMAC and 13.9 M SiLU elements per 640×640 image (315 MACs per activation).

IV. Results

A. Instruction count, CPI and speed (E1)

Table I and Fig. 1 show the headline loop. The custom instruction executes 6.0 instructions per activation including 5 of loop overhead, against 12.8 for integer f_4, 21.0 for the table and 114 for `expf`. Out-of-order execution lowers cycles per activation by 4.6× (`expf`), 4.3× (integer f_4) and 6.8× (instruction); with it the instruction loop reaches 1.03 cycles, i.e. the front end of the core, not the SiLU unit, is the limit. The in-order result (7.0) is limited by the 5-instruction loop at CPI 1.17.

Speedups of the instruction are 24.1× over `expf`, 2.35× over integer f_4 and 3.0× over the table in-order; 35.9×, 3.72× and 4.9× out-of-order. The "10.8×" headline of earlier versions of this project was a loop-level number against the weakest baseline and should not be quoted as a system speedup. CPI *falls* on the out-of-order core (0.17 for the instruction) but CPI alone misleads: the instruction raises CPI in-order relative to the table (1.17 vs 1.00) while still being

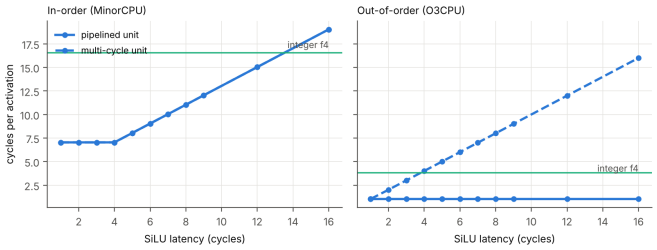


Fig. 2. Cycles per activation against unit latency (1 GHz core, instruction loop only).

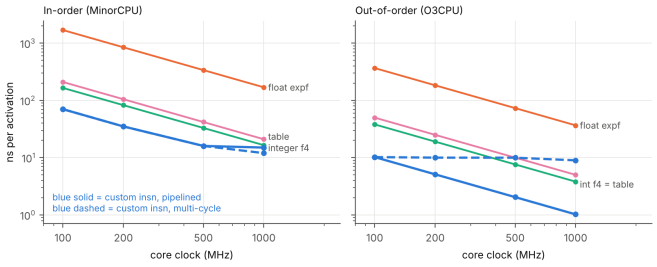


Fig. 3. Wall-clock time per activation against clock for each core and unit style.

3× faster, because it removes instructions. The iron law must be read with all three terms.

B. Latency, clock and pipelining (E2, E2b)

Fig. 2 varies latency from 1 to 16 cycles. In-order, cycles are flat up to latency 4 (the first few cycles are absorbed by the remaining loop instructions) and then grow one-for-one with latency; pipelined and unpipelined units give identical values, since an in-order core does not issue the next SiLU before the previous result is consumed. Out-of-order, a *pipelined* unit hides every latency tested (1.03 cycles at 16), while an *unpipelined* unit exposes it completely: 5 cycles at 500 MHz and 9 at 1 GHz. At 500 MHz an unpipelined unit is therefore *slower* than integer f_4 (0.76× of its speed); at 1 GHz it is slower (0.42×). This is a structural hazard in the sense of the course, and it implies that the 8.058 ns unit must be pipelined to be useful in a fast out-of-order core.

Fig. 3 converts to time. Faster clocks raise the latency in cycles, so at 1 GHz the in-order core is only 1.1× (12-stage pipelined unit) to 1.4× (9-cycle multi-cycle unit) faster than integer f_4. Clocks above ≈123 MHz extrapolate an FPGA delay to faster logic and are illustrative only.

C. Branch prediction (E4)

Software f_4 is a compare chain. With sorted inputs the branches are perfectly predictable (0.005 mispredicts per activation), which flatters the software baseline. With shuffled inputs (a fixed xorshift permutation) it mispredicts 0.41 branches per activation (32 per 1000 instructions), costing +3.4 cycles in-order (16.6→20.0) and +5.4 out-of-order (3.8→9.2). `expf` adds 0.44 mispredicts per activation (+5.7 and +4.8 cycles).

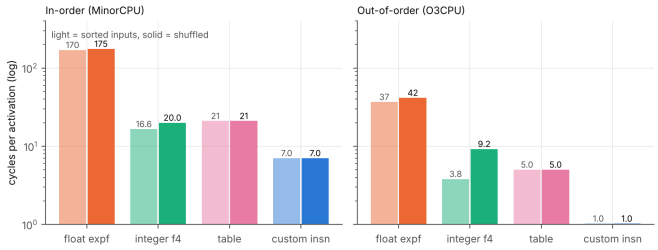


Fig. 4. Software f_4 and `expf` with sorted and shuffled inputs.

TABLE II
ALIGNMENT SENSITIVITY OF THE LOOP (CYCLES/ACTIVATION, 100 MHZ). MIN–MAX OVER FIVE ALIGNMENTS.

Variant	In-order	Out-of-order
float `expf`	169.8–169.8	36.6–36.9
integer f_4	16.6–18.1	3.8–5.7
table	20.0–21.0	5.0–6.0
custom insn	7.0–9.0	1.0–4.0

The table and the custom instruction have no data-dependent branch and are unaffected (Fig. 4). Against shuffled inputs, the instruction is 2.8× (in-order) and 9.0× (out-of-order) faster than integer f_4; the order-of-magnitude advantage of the out-of-order result comes in large part from removing branch mispredictions.

D. Code-placement sensitivity (E7)

Tiny loops in gem5 are front-end sensitive. Compiling the same source with different loop alignments moves the out-of-order instruction loop between 1.03 and 4.02 cycles per activation and integer f_4 between 3.82 and 5.69 (Table II); inside the convolution layer the extra cost of an activation varies by more than 10 cycles. This is an artefact of the simulated front end, not of the instruction, and sets the resolution of this study: differences of ±1–3 cycles per activation between the cheap variants are not conclusions. Only `expf` is cleanly separated in the conv layer.

E. Cache pressure (E6)

A table needs data memory; the instruction does not. Shrinking the L1 data cache from 64 kB to 512 B (working set 2 kB of inputs plus 0.5 kB of table) costs the table 35% in-order (21.0→28.5) and 21% out-of-order (5.0→6.1); the instruction moves from 8.0 to 8.5 and from 1.0 to 1.5 (its only loads are the inputs). Once the cache is 1 kB or larger the table loses at most 3% in-order, so at this table size the instruction's advantage over the table is mostly the instruction count, not the cache. A bigger table would suffer sooner.

F. Whole model: YOLOv8n and Amdahl's law (E5)

Accuracy. Table III gives COCO128 accuracy. Most of the loss is fixed-point rounding itself (−0.0066 mAP50–95); f_4 adds about 0.0015, well below the noise of a 128-image set. 8.8% of YOLOv8n's activation inputs lie outside $[-4, 4]$

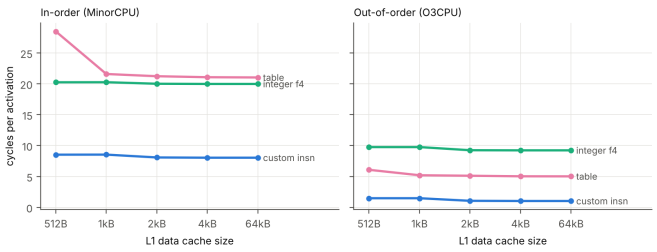


Fig. 5. Cycles per activation against L1 data-cache size (500 MHz, shuffled inputs, no L2).

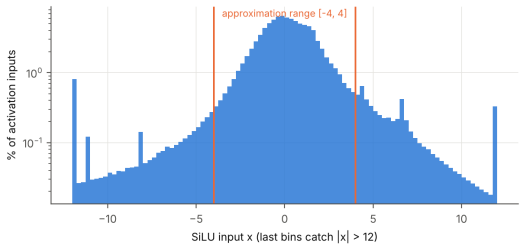


Fig. 6. Distribution of YOLOv8n activation inputs and the $[-4, 4]$ range.

TABLE III
YOLOv8n on COCO128 (relative comparison only). 57/57 SiLU modules replaced in each row.

Activation	mAP50	mAP50–95	Δ
exact SiLU	0.6024	0.4439	—
exact, Q6.10 rounding	0.5924	0.4374	−0.0066
f_4 (16-bit strict)	0.5991	0.4359	−0.0081
f_4 (exact tails)	0.6007	0.4359	−0.0080
257-entry table	0.5958	0.4373	−0.0066

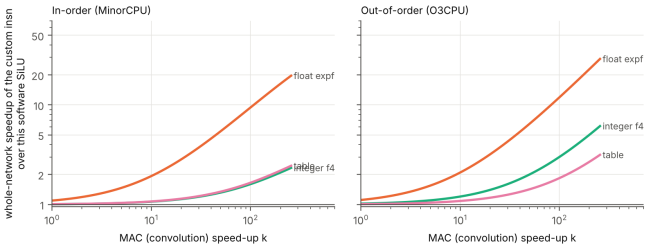


Fig. 7. Estimated whole-YOLOv8n speedup against MAC acceleration k (500 MHz).

(Fig. 6), the region where a segment-based approximation relies on its tail rules, so restricting evaluation to $[-4, 4]$ ignores a non-trivial share of real inputs. The micro-benchmark sweeps $[-4, 4)$, where the six segments are active; the model run exercises the tails as well.

Speed. The convolution layer measures cycles per MAC (median over five alignments: 5.67 in-order, 1.15 out-of-order at 500 MHz); the shuffled-loop experiment gives cycles per activation. With 315 MACs per activation in YOLOv8n,

$$S = \frac{t_{MAC}/k + t_{act}^{base}}{t_{MAC}/k + t_{act}^{insn}} \tag{2}$$

where t_{MAC} and t_{act} are the whole-network MAC and activation times (MACs × cycles per MAC; activations × cycles per activation) and k is the speedup applied to the MAC part (SIMD, accelerator); this is Amdahl's law with the activation share $p = t_{act}^{base}/(t_{MAC}/k + t_{act}^{base})$. Fig. 7 and Table IV give the estimate. On a scalar core the activation is 8.7% (in-order) to 9.9% (out-of-order) of run time with `expf` and under 2% with integer f_4, so the custom instruction gives only 1.09–1.11× over `expf` and 1.01–1.02× over integer f_4. If MACs are 16× faster, the figures rise to 2.5–2.7× and 1.1–1.3×. The instruction matters when the multiply-accumulate part is accelerated, not on a scalar core — a direct, quantitative Amdahl result.

V. Discussion

What the instruction buys. It replaces about 13 integer instructions and a compare chain by one operation. That is a large win against a library `expf` (24–36× on the loop) and a moderate one against well-written integer code (2.4–3.7×, more with unpredictable data). The honest system-level statement is that a scalar RISC-V core running YOLOv8n gains about 10% over `expf` and about 1–2% over integer f_4; the instruction becomes valuable only in a balanced design in which the MACs are already accelerated.

What the microarchitecture decides. The same instruction is limited by dependence latency in order, by issue rate and unit pipelining out of order, and by branch behaviour in its software competitor. The iron-law terms move in different directions with the core: IC always falls, CPI falls or rises, and T_{clk} is set by the unit's pipeline depth. Because the out-of-order core overlaps iterations, a pipelined unit makes latency nearly irrelevant, whereas a unit that cannot accept one operation per cycle destroys the benefit.

Accuracy cost. f_4 is a coarse approximation (mean error 0.028, about 60× the table's) but its effect on detection accuracy beyond Q6.10 rounding is within the noise of our test set. This does not prove equivalence on the full COCO validation set.

VI. Threats to Validity

- One activation loop and one scalar convolution layer; the network-level numbers are a scaled estimate, not a simulation of YOLOv8n.
- Alignment sensitivity of ±1–3 cycles (Table II); the cheap variants cannot be ranked inside the conv layer.
- The 8.058 ns delay is from a slow-corner FPGA; 0.3 ns per pipeline stage is an assumption; no clock uncertainty was in the timing constraints; clocks above 123 MHz are extrapolations.
- Default in-order and out-of-order gem5 models, one SiLU unit, no L2 in E6; compiler is clang with musl `expf`, not GCC with glibc, which would change the `expf` baseline.
- COCO128 is a small subset of train2017, used only for comparing activations.

TABLE IV
ESTIMATED WHOLE-NETWORK SPEEDUP OF THE CUSTOM INSTRUCTION (YOLOV8N, 500 MHZ).

	vs `expf`		vs integer f_4	
k	In-order	O3	In-order	O3
1	1.09	1.11	1.01	1.02
4	1.37	1.44	1.03	1.08
16	2.48	2.75	1.10	1.32
64	6.60	8.00	1.40	2.28

- No power or area comparison with a floating-point SiLU unit.
- Not done: GPU comparison (e.g. Vortex) and a full-system run of YOLOv8n in gem5.

VII. CONCLUSION

A multiplier-free SiLU instruction is a 24–36× win on the activation loop against `expf` and a 2.4–3.7× win against integer C, but a 1.1× and 1.01–1.02× win respectively on a scalar-core YOLOv8n; its value grows with how much the rest of the network is accelerated. It must be pipelined to help an out-of-order core, and its advantage over software is larger for unpredictable data because it removes branches. The flow that derives its latency from synthesis is scripted and reproducible, with all raw runs, tables and figures kept in the repository `normie2099/SiLU-Architecture`.

REPRODUCIBILITY

All numbers are produced by commands in `docs/REPRODUCE.md`; raw gem5 outputs are in `results/raw/` and the tables and figures in this report are generated from them by `scripts/make_results.py`. 296 gem5 runs were made.

REFERENCES

[1] F. Spagnolo, S. Perri and P. Corsonello, "Approximate Swish activation function for low-energy yet low-error VLSI implementations," in *Proc. IEEE Computer Society Annual Symp. on VLSI (ISVLSI)*, 2025, doi:10.1109/ISVLSI65124.2025.11130259.

[2] S. Elfwing, E. Uchibe and K. Doya, "Sigmoid-weighted linear units for neural network function approximation in reinforcement learning," *Neural Networks*, vol. 107, pp. 3–11, 2018.

[3] P. Ramachandran, B. Zoph and Q. V. Le, "Searching for activation functions," arXiv:1710.05941, 2017.

[4] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.

[5] J. Lowe-Power *et al.*, "The gem5 simulator: Version 20.0+," arXiv:2007.03152, 2020.

[6] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[7] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conf.*, 1967, pp. 483–485.

[8] G. Jocher, A. Chaurasia and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[9] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International.